

6 – SEQUENCES

A sequence is an ordered collection of items that supports operations such as indexing, slicing, iteration and membership testing. They form a core part of the python language and many built-in types are sequences.

Sequences may be categorized into two main groups: Immutable and mutable.

Immutable

Immutable types include `str`, `tuple` and `bytes`. Variables of these does not change after initialization. When a variable of these type is changed, the new value is assigned to a new location. The existing variable reference is changed to point to the new location.

Information

The numeric types `int` and `float` are immutable as well as the set type `frozenset`.

```
# Demonstrates immutability
a = 'hello'
b = a
print(f'{a:<20} {b}')
hello                hello

print(f'{id(a):<20} {id(b)}')
1389813655632        1389813655632

a = 'world'           # a new value is assigned to a
print(f'{a:<20} {b}')
world                hello

print(f'{id(a):<20} {id(b)}')
1389818780176        1389813655632
```

Mutables

Mutable types include `list` and `bytearray`, and most custom type (types that are user defined). When a variable of these types is changed, the new value is assigned to the existing location.

Information

The mapping type `dict` is immutable as well as the set type `set`.

```
# Demonstrates mutability
a = [1]
b = a
print(f'{a} {b}')
[1] [1]

print(f'{id(a):<20} {id(b)}')
1389824700992      1389824700992

a.append(2)
print(f'{a} {b}')
[1, 2] [1, 2]

print(f'{id(a):<20} {id(b)}')
1389824700992      1389824700992
```

6.a – String

A string is a sequence of characters that is delimited with either a pair of single quotes, double quotes or triple single or triple double quotes. In a string literal, you may not use mis-match quotes as your delimiter. i.e. the start and end delimiters must be identical in a literal definition.

Strange fact

Python does not have a native `char` data type as does most modern languages.

```
a = 'Hello '
b = "world"
```

```
c = '''This string  
can span multiple  
lines.'''
```

6.a.1 – String Creation Methods

The `str()` function can create a string from pretty much any literal. There are also lots of functions that returns a string.

```
>>> str('Hello World')  
'Hello World'  
  
>>> str(3.14159)  
'3.14159'  
  
>>> str([1, 2])  
'[1, 2]'  
  
>>> str(True)  
'True'  
  
>>> str(None)  
'None'
```

6.a.1 – String Concatenation

You can add strings to string to create a new string. This is called concatenation and is supported by most modern high-level language. It results in a new string. In python you are able to multiple a string by an integer.

Fun fact

Ruby and PERL also allow you to multiple a string by an integer.

```
>>> 'Hello ' + 'Narendra'  
'Hello Narendra'  
  
>>> 'Hi ' * 3  
'Hi Hi Hi '
```

6.a.2 – Escaping a String

Because quotations are special, (it denotes the start and end of a string) if you want to use a quote in a string, you need to escape it with a `\`

```
>>> 'She said \'hi\''
"She said 'hi'"

>>> print('Hi\nNarendra')           #embedding a newline character
Hi
Narendra

>>> print('Hi\tNarendra')           #embedding a tab
Hi      Narendra
```

6.a.3 – Raw strings

All string in python are checked for meta-characters such as `\n` (newline) and `\t` (tab). As you may notice, these sequences are prefixed with the backslash character (`\`). If you want to use the backslash as part of your string, then you have to prefix it with another backslash. Thus, `print('\\/n')` with output

`'/n'`. This can be tedious to type and hard to read, so the language designers came up with a technique that allows a programmer to indicate to the interpreter that a string should not be processed. Raw strings are especially convenient when using regular expressions.

Fun fact

Other languages that support raw strings are C#, C++ and Rust. Java does not support raw strings.

A prefix of `r` (or `R`) before a string tells python that the string should not be processed.

```
>>> print(r'\n')                     #raw string
\n

>>> print('\n')                     #normal string

>>> print('\\n')                     #normal string
\n

>>> print(r'\\n')                     #raw string
\\n
```

```
>>>
```

6.a.4 – Combining F-Strings and raw strings

It is possible to embed an expression within a raw string as the example below shows:

```
>>> base = 'facultyweb.centennialcollege.ca'
>>> print(rf'http://{base}/npersh')
http://facultyweb.centennialcollege.ca/npersh
```

6.b – List

A list is a collection of values that can be of any type. It is mutable i.e., it is possible to modify an existing list. You may perform indexing and slicing on a list.

6.b.1 – Creating Lists

You may create list using the following statements:

```
a = list()
b = []
c = [9, 8, 7]
d = 'Mary had a little lamb'.split()
e = [0, 'one', [2], 3.0, True]
```

6.b.2 – Adding Lists Together

You can extend a list with another list by using the extend keyword, or the + symbol.

```
list1 = [1, 2, 3]
list2 = [9, 8, 7]
new_list = list1 + list2
list1 += list2
```

6.b.3 – Adding an Item to a List

If you just want to add a single item to a list, you need to use the `append()` method.

```
fruits = ['apple', 'banana', 'orange']
fruits.append('pear')
```

The in operator

This operator returns True if the item exists in the list.

```
colors = ['red', 'green', 'blue']
color = 'white'
print(f'{color in colors}')      #display False
```

List Indexing

To get hold of a particular item from a list you can use its index number. This number can also be negative, if you want to start counting from the end of the list.

```
letters = ['a', 'b', 'c']
letters[0]          #give 'a'
letters[-1]         #the last item 'c'
letters[0] = 'z'    #sets the first item to 'z'
```

List Slicing

Using the list index and the colon symbol you can slice up a list to get only the portion you want. Start is included, but end is not.

```
#list[start:end:step]
letters = ['a', 'b', 'c', 'd']
letters[1:3]      #Result: ['b', 'c']
```

6.c – Common Sequence Operations

The following operations can be performed on all sequences.

6.b.1 – min(), max() sum()

6.b.1 – Length

6.b.1 – Membership test

6.b.1 – Concatenation

6.b.1 – Repetition

6.b.1 – Iteration

6.b.1 – Indexing

6.b.1 – Slicing